

PRACTICAL WEEK 8

TASK1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
GNU nano 7.2 task1.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc()
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("Memory allocated using malloc:\n");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);

    printf("\n");

    // calloc()
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("Memory allocated using calloc:\n");
```

```
GNU nano 7.2 task1.c *
    }

    printf("Memory allocated using calloc:\n");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);

    printf("\n");

    // realloc()
    a = (int *)realloc(a, 10 * sizeof(int));

    if (a == NULL)
    {
        printf("realloc failed\n");
        free(b);
        return 1;
    }

    for (i = 5; i < 10; i++)
        a[i] = i + 1;

    printf("Memory after realloc:\n");
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);

    printf("\n");

    // free()
    free(a);
    free(b);

    printf("Memory freed successfully.\n");

    return 0;
}
```

```
jhansi_priya@Jhansi-Priya: ~/ x + v
jhansi_priya@Jhansi-Priya:~$ mkdir Practical8
jhansi_priya@Jhansi-Priya:~$ cd Practical8
jhansi_priya@Jhansi-Priya:~/Practical8$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Practical8$ gcc task1.c -o task1
jhansi_priya@Jhansi-Priya:~/Practical8$ ./task1
Memory allocated using malloc:
1 2 3 4 5
Memory allocated using calloc:
0 0 0 0 0
Memory after realloc:
1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
```

TASK2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
jhansi_priya@Jhansi-Priya:~/Practical8$ nano task2.c
jhansi_priya@Jhansi-Priya:~/Practical8$ gcc task2.c -o task2
jhansi_priya@Jhansi-Priya:~/Practical8$ ./task2
Parent PID: 665
Memory allocated: 10 MB
Before fork()
Press Enter to continue...

Parent continues running.
Child PID: 705
Parent has not modified the memory.
Press Enter after checking memory usage...

Child PID: 705
Parent PID: 665
Child has started.
Press Enter before modifying memory...

^C
jhansi_priya@Jhansi-Priya:~/Practical8$
```

```
jhansi_priya@Jhansi-Priya: ~ x + v - □ X
jhansi_priya@Jhansi-Priya:~$ grep -E "VmRSS|VmSize" /proc/665/stat
VmSize: 12932 kB
VmRSS: 11844 kB
```

```
jhansi_priya@Jhansi-Priya: ~/  ×  +  ∨
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (10 * 1024 * 1024)

int main()
{
    char *data;
    pid_t pid;

    // Allocate 10 MB
    data = (char *)malloc(SIZE);

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    // Write to all pages so memory is actually allocated
    for (size_t i = 0; i < SIZE; i += 4096)
    {
        data[i] = 'A';
    }

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated: 10 MB\n");

    printf("Before fork()\n");
    printf("Press Enter to continue...\n");
    getchar();

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
```

```
jhansi_priya@Jhansi-Priya: ~/  ×  +  ∨
GNU nano 7.2 task2.c *

    {
        // Child process
        printf("\nChild PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Child has started.\n");
        printf("Press Enter before modifying memory...\n");
        getchar();

        // Modify the memory
        for (size_t i = 0; i < SIZE; i += 4096)
        {
            data[i] = 'B';
        }

        printf("Child modified the memory.\n");
        printf("Press Enter to exit child...\n");
        getchar();

        free(data);
        exit(0);
    }
    else
    {
        // Parent process
        printf("\nParent continues running.\n");
        printf("Child PID: %d\n", pid);

        printf("Parent has not modified the memory.\n");
        printf("Press Enter after checking memory usage...\n");
        getchar();

        wait(NULL);

        printf("Child finished.\n");

        free(data);
    }

    return 0;
}
```


